

COMPUTER SCIENCE 3:

Individual Activity on Python Classes and Objects

Name: FRANKCHESKA PLOTEA

Year & Section: 9-LITHIUM

Instruction: Download this worksheet, rename it as FA_FamilyName and complete the tasks. Submit this in PDF format in the [GForm](#) provided.

This activity will guide you through the core concepts of Python Object-Oriented Programming (OOP), transitioning from conceptual class design via UML diagrams to writing functional code.

Introduction to Python OOP & Class Diagrams

Grade Level: 9

Topic: Classes, Objects, and UML Class Diagrams in Python

Learning Objectives

By the end of this lesson, students will be able to:

1. Define a **Class** as a blueprint and an **Object** as an instance of that class.
2. Interpret and draw a basic **UML Class Diagram** showing attributes and methods.
3. Write Python code to define a class with an `__init__` constructor and instance methods.
4. Instantiate objects and call their methods in Python.

Part 1: Review/Recall (10 Mins)

Recall our previous (last meeting) discussion on OOP basics: Classes, Objects, and the UML Class Diagram. You may refer to and go through **SG 4 and 5** in our Computer Science 3 KHub.

1. The Blueprint Analogy

- **Class:** A blueprint or template. It defines what attributes (data) and behaviors (actions) an entity will have, but it isn't the physical thing itself. Example: A blueprint for a house.
- **Object:** An individual instance created from that blueprint. Example: The actual physical house built at 123 Main Street.

2. UML Class Diagram Structure

Before coding, programmers design classes using **UML (Unified Modeling Language) Class Diagrams**. A standard single-class diagram uses a box divided into three sections:

Python

```
+-----+
|           className           | <-- Top: Name of the class (PascalCase)
+-----+
| - attribute1: dataType        | <-- Middle: Attributes (Variables/Data)
| - attribute2: dataType        |
```

```
+-----+
| + method1(param): returnType      | <-- Bottom: Methods (Functions/Actions)
| + method2(): returnType           |
+-----+
```

3. Translating UML to Python Code

Here is how a UML class diagram translates directly into Python syntax:

Python

Python

```
class ClassName:
    def __init__(self, attribute1, attribute2):
        # Initializing attributes (Constructor method)
        self.attribute1 = attribute1
        self.attribute2 = attribute2

    def method1(self, param):
        # Action/Behavior
        pass
```

Key Python concepts to emphasize:

- **class keyword:** Used to define a new class.
- **__init__() method:** The constructor function that runs automatically when a new object is created.
- **self parameter:** Refers to the specific object instance being worked with. It must be the first parameter in `__init__` and all instance methods.

Part 2: Hands-On Activity & Guided Problem (30 Mins)

Problem Scenario: The Digital Pet Simulator

TASK: You will design and program a class for a virtual pet simulator called `VirtualPet`.

Requirements:

Each pet must track:

- `name` (text string)
- `species` (text string)
- `energy` (integer, starting at 100)

Each pet can perform the following actions:

- `play()`: Reduces energy by 20 and prints a message saying the pet played.
- `rest()`: Increases energy by 15 and prints a message saying the pet rested.
- `get_status()`: Prints the pet's current energy level.

Step 1: Design the Class Diagram

Instruction: Create a UML Class Diagram for the Digital Pet Simulator. Also refer to the specific requirements provided.

Step 2: Implement the Python Code

Refer to the Python code provided, which should match the UML Class Diagram you created. The Python code has missing elements, indicated as `-----`; identify them to make the code work.

You may save your code in your own folder or GitHub for personal reference.

Python

Python

```
class VirtualPet:
    def __init__(self, name, species):
        # Attributes
        self.name = name
        self.species = species
        self.energy = 100 # Default starting energy

    def play(self):
        """Reduces energy when the pet plays."""
        if self.energy >= 20:
            self.energy -= 20
            print(f"{self.name} the {self.species} played happily! Energy is now {self.energy}.")
        if self.energy < 20:
            print(f"{self.name} is too tired to play! Rest first.")

    def rest(self):
        """Restores energy when the pet rests."""
        self.energy += 15
        if self.energy > 100:
            self.energy = 100 # Cap maximum energy at 100
        print(f"{self.name} rested. Energy restored to {self.energy}.")

    def get_status(self):
        """Displays the pet's current state."""
        print(f"--- {self.name}'s Status ---")
        print(f"Species: {self.species}")
        print(f"Energy: {self.energy}/100\n")

# --- Main Program / Object Instantiation ---

# 1. Create two pet objects from the VirtualPet class
```

```
pet1 = VirtualPet("Milo", "Dog")
pet2 = VirtualPet("Luna", "Cat")
```

```
# 2. Interact with pet1 (Milo)
```

```
pet1.get_status()
pet1.play()
pet1.play()
pet1.rest()
```

```
# 3. Interact with pet2 (Luna)
```

```
pet2.get_status()
pet2.play()
```

Part 3: Wrap-Up & Knowledge Check (10 Mins)

Answer the following questions; cite your sources if necessary:

1. What is the difference between a class and an object?

A class and an object are the two main components in Object-Oriented Programming that rely on one another to create real life objects. Though they work together, the main difference between the two is that classes provide the blueprint to creating objects while objects are instantiated from classes. Classes provide the framework to define what an object should do and be like and objects are the result of the datum in classes.

2. What role does `__init__` play in a Python class?

The `__init__` keyword acts as the indicator for a program to run automatically without being called as soon as an object has been made.

3. Why is `self` included in method definitions?

Self is an important method in definitions because it refers to what object the program is working with and what the attributes and methods are being directed to.